

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:40

Annexure A

Industry Problem Solving Task

Code of Conduct:

I ____Delisha A K(192511307)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

ANSWER:

Introduction

The Smart Banking and Financial Management System is designed to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The system follows a **layered object-oriented architecture** consisting of the Presentation Layer, Service Layer, Domain Layer, and Data Access Layer. These layers are connected to an **Object-Oriented Database Management System (OODBMS)** for storing and retrieving persistent banking objects.

The system uses the **Factory Pattern**, **Strategy Pattern**, **Observer Pattern**, and **Repository Pattern** to improve flexibility, scalability, security, and maintainability.

Object-Oriented Principles

Encapsulation

Encapsulation combines data and methods within a class and protects important banking information from unauthorized access.

For example, a `BankAccount` class can contain account number, account balance, account type, and customer details. Methods such as `deposit()`, `withdraw()`, and `transfer()` can control how the account information is modified.

This is especially important in banking because sensitive financial data must be protected.

Abstraction

Abstraction hides complex internal operations and provides only the required functionality to the user.

For example, when a customer performs a fund transfer, the customer only selects the account and enters the amount. The internal validation, transaction processing, and database operations are hidden from the user.

This reduces complexity and makes the system easier to use.

Inheritance

Inheritance allows common properties and operations to be reused between related classes.

For example:

Account → **SavingsAccount**

Account → **CurrentAccount**

Account → **LoanAccount**

The common properties of an account can be defined in the `Account` class, while specialized behaviour can be implemented in the derived classes.

This reduces code duplication and improves reusability.

Polymorphism

Polymorphism allows the same operation to behave differently for different objects.

For example, interest calculation can work differently for different account or loan types. Each type can provide its own implementation while using the same general operation.

This makes the banking system flexible and easier to extend.

Layered Architecture

The system consists of four major layers.

Layer	Responsibility
Presentation Layer	Provides the user interface and receives user requests
Service Layer	Processes banking operations and coordinates services

Domain Layer	Contains banking objects and business rules
Data Access Layer	Communicates with the OODBMS and manages persistent objects

Presentation Layer

The Presentation Layer provides the interface through which customers and bank employees interact with the system.

It can provide functions such as:

- Viewing account balance
- Transferring funds
- Applying for loans
- Paying bills
- Viewing transaction history

The Presentation Layer sends requests to the Service Layer instead of directly accessing the database.

Service Layer

The Service Layer handles the main banking operations.

For example, during a fund transfer, it can:

- Validate the request
- Check account information
- Apply transaction rules
- Coordinate the transfer
- Request data from the Data Access Layer

This prevents business logic from being placed directly inside the user interface.

Domain Layer

The Domain Layer represents the main banking objects and business rules.

Important objects can include:

- Customer
- Account
- Transaction
- Loan
- Bill
- Payment

This layer represents how the banking system actually works.

Data Access Layer

The Data Access Layer communicates with the OODBMS.

It performs operations such as:

- Saving account objects
- Retrieving customer objects
- Updating transactions
- Storing loan information
- Retrieving transaction history

This separation makes database-related changes easier to manage.

Design Patterns Used

Factory Pattern

The Factory Pattern is used to create different types of bank accounts.

For example:

Account Factory → Savings Account / Current Account / Loan Account

The application does not need to directly create every account object. Instead, the factory determines and creates the required object.

Advantages:

- Simplifies object creation
- Reduces dependency between classes
- Supports new account types
- Improves maintainability

Strategy Pattern

The Strategy Pattern is used for different **loan-interest and payment calculation policies**, as specified in the problem statement.

Different strategies can be created for different calculation policies.

For example:

Loan Calculation → Fixed Interest Strategy

Loan Calculation → Variable Interest Strategy

The calculation method can be changed without changing the main loan-processing system.

Advantages:

- Provides flexibility
- Allows algorithms to be changed easily
- Supports different banking policies
- Reduces modification of existing code

Observer Pattern

The Observer Pattern is used for **transaction and fraud alerts**.

When a transaction occurs, interested components can automatically receive a notification.

For example:

Transaction → Notification Service → Customer

If a suspicious transaction is detected:

Transaction → Fraud Alert → Customer/Security System

Advantages:

- Supports real-time notifications
- Improves fraud awareness
- Reduces dependency between components
- Allows additional notification services to be added easily

Repository Pattern

The Repository Pattern manages communication between the business components and persistent banking objects.

For example, an AccountRepository can provide operations such as:

- saveAccount()
- findAccount()
- updateAccount()
- deleteAccount()

The Service Layer does not need to directly handle database operations.

Advantages:

- Separates business logic from database logic
- Simplifies data access
- Improves maintainability
- Makes testing easier

Communication with the OODBMS

The OODBMS stores banking objects in persistent form. The Data Access Layer and Repository Pattern act as the connection between the application and the database.

The overall communication can be represented as:

Customer → Presentation Layer → Service Layer → Domain Layer → Data Access Layer → OODBMS

Storing Banking Objects

When a new account is created:

- The customer enters account details through the Presentation Layer.

- The Service Layer receives and validates the request.
- The Factory Pattern creates the appropriate account object.
- The Domain Layer applies the required banking rules.
- The Repository sends the object to the Data Access Layer.
- The Data Access Layer stores the object in the OODBMS.

Retrieving Banking Objects

When a customer requests account information:

- The request is received by the Presentation Layer.
- The Service Layer processes the request.
- The Repository requests the required object.
- The Data Access Layer retrieves it from the OODBMS.
- The object is returned to the Service Layer.
- The required information is displayed through the Presentation Layer.

This provides a clear separation between application logic and persistent data management.

Example of Fund Transfer

A fund transfer demonstrates how the different layers and patterns work together.

Customer

↓

Presentation Layer

↓

Service Layer

↓

Domain Layer

↓

Repository / Data Access Layer

↓

OODBMS

The system retrieves the sender and receiver account objects from the OODBMS. The Service Layer validates the transaction, while the Domain Layer checks the relevant banking rules such as account status and available balance.

After the transfer is completed, the updated account and transaction objects are stored back in the OODBMS.

The Observer Pattern can then generate a transaction notification for the customer. If the transaction is suspicious, a fraud alert can also be generated.

Scalability, Security, Flexibility and Maintainability

Requirement	Contribution of the Design
Scalability	New banking services can be added without redesigning the entire system
Security	Encapsulation and layered access help protect sensitive banking information
Flexibility	Strategy and Factory Patterns allow new policies and account types to be introduced
Maintainability	Separation of responsibilities makes individual layers easier to modify
Reusability	Common classes and services can be reused
Extensibility	New services such as investment management and digital wallets can be integrated

Future Enhancements

The architecture is designed to support future services such as **investment management and digital wallets**.

New domain objects and services can be added without changing the complete application. New strategies can also be introduced for investment calculations or digital-wallet payment policies. This makes the system suitable for continuous development as banking requirements change.

Conclusion

The Smart Banking and Financial Management System demonstrates how **object-oriented principles, layered architecture, and design patterns** can be combined to create a reliable banking application.

Encapsulation and abstraction help protect data and reduce complexity, while inheritance and polymorphism improve reusability and flexibility. The layered architecture separates presentation, service, domain, and data access responsibilities.

The Factory Pattern simplifies object creation, the Strategy Pattern provides flexible calculation policies, the Observer Pattern supports transaction and fraud alerts, and the Repository Pattern provides organized access to persistent banking objects.

The OODBMS stores and retrieves banking objects through the Data Access Layer. Therefore, the proposed architecture provides a **scalable, secure, flexible, maintainable, and extensible solution** that can support future banking services such as investment management and digital wallets.

2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

ANSWER:

Introduction

The Smart Retail and Inventory Management System is developed to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a **layered object-oriented architecture** consisting of the Presentation Layer, Business Layer, Domain Layer, and Data Access Layer connected to an **Object-Oriented Database Management System (OODBMS)**.

The system uses the **MVC Pattern, Factory Pattern, Observer Pattern, and DAO Pattern**. These concepts improve the flexibility, scalability, maintainability, and efficiency of the retail system.

Object-Oriented Design Principles

Encapsulation

Encapsulation combines data and methods within a class and protects important retail information.

For example, a Product class can contain product ID, product name, price, quantity, and category. Methods such as updateStock() and updatePrice() can control how the product information is modified.

This prevents direct and incorrect modification of inventory data.

Abstraction

Abstraction hides unnecessary implementation details from users and other components.

For example, a store manager can select **Update Stock** without knowing how the system validates the quantity or stores the updated product object in the OODBMS.

This reduces complexity and makes the application easier to use.

Inheritance

Inheritance allows common properties and behaviours to be reused.

For example:

Product → Electronics

Product → Clothing

Product → Grocery

The common product information can be maintained in the Product class, while each category can have its own specialized properties.

This reduces code duplication and improves reusability.

Polymorphism

Polymorphism allows the same operation to behave differently depending on the product type.

For example, different product categories can have different pricing, discount, or stock-handling rules while using a common product interface.

This makes it easier to introduce new product categories in the future.

Layered Architecture

The system consists of four major layers.

Layer	Responsibility
Presentation Layer	Provides interfaces for managers, employees, and customers
Business Layer	Performs sales, stock, purchasing, and inventory operations
Domain Layer	Represents products, suppliers, customers, orders, and stock
Data Access Layer	Communicates with the OODBMS for persistent inventory data

Presentation Layer

The Presentation Layer provides the user interface.

Users can perform operations such as:

- Add or update products
- View stock levels
- Process sales
- Manage suppliers
- Create purchase orders
- View inventory information

The Presentation Layer sends requests to the Business Layer.

Business Layer

The Business Layer contains the main business operations of the retail system.

It handles activities such as:

- Processing sales
- Managing stock
- Managing purchase orders

- Checking minimum stock levels
- Managing suppliers
- Applying business rules

This layer coordinates the different operations without directly handling database storage.

Domain Layer

The Domain Layer represents the important objects in the retail system.

Examples include:

- Product
- Supplier
- Customer
- PurchaseOrder
- SalesTransaction
- WarehouseStock

These objects contain the data and business rules related to the retail system.

Data Access Layer

The Data Access Layer manages communication with the OODBMS.

It performs operations such as:

- Storing product objects
- Retrieving stock information
- Updating inventory
- Saving sales transactions
- Retrieving supplier and purchase-order information

This separation makes the database operations easier to maintain.

Design Patterns Used

MVC Pattern

The **MVC Pattern** is used to manage the user interface of the retail system.

MVC consists of:

Model: Represents the retail data and objects.

View: Displays information to the user.

Controller: Receives user requests and coordinates the required operations.

For example, when a manager wants to update product stock:

Manager → View → Controller → Model → Database

The MVC Pattern separates the user interface from the application logic.

Benefits:

- Organizes the user interface
- Separates responsibilities
- Makes the system easier to maintain
- Allows changes to the interface without changing the complete business logic

Factory Pattern

The Factory Pattern is used for creating different product categories.

For example:

Product Factory → Electronics / Clothing / Grocery

The factory determines which product object should be created.

Benefits:

- Simplifies object creation
- Reduces dependency between classes
- Supports new product categories
- Improves maintainability

Observer Pattern

The Observer Pattern is used to notify managers when inventory reaches the minimum threshold.

For example:

Stock Level → Minimum Threshold Reached → Manager Notification

When the stock quantity becomes lower than the defined minimum level, the system automatically sends an alert.

This helps managers take action before the product becomes unavailable.

Benefits:

- Provides automatic inventory alerts
- Supports real-time monitoring
- Reduces manual checking
- Helps prevent stock-outs

DAO Pattern

The **Data Access Object (DAO) Pattern** handles communication with persistent inventory objects.

For example, a ProductDAO can provide operations such as:

- saveProduct()
- findProduct()
- updateProduct()
- deleteProduct()

The Business Layer can use these operations without directly dealing with the OODBMS.

Benefits:

- Separates data access from business logic
- Makes database operations easier to manage

- Improves maintainability
- Supports testing and future database changes

Interaction Between Layers and OODBMS

The different layers communicate in a controlled sequence.

User → Presentation Layer → Business Layer → Domain Layer → Data Access Layer → OODBMS

Storing Inventory Data

When a new product is added:

- The user enters the product details through the Presentation Layer.
- The Controller processes the request.
- The Business Layer validates the product information.
- The Factory Pattern creates the appropriate product object.
- The Domain Layer represents the product.
- The DAO sends the object to the Data Access Layer.
- The Data Access Layer stores the persistent object in the OODBMS.

Retrieving Inventory Data

When a manager wants to view stock information:

- The request is received through the Presentation Layer.
- The Business Layer processes the request.
- The DAO requests the required inventory object.
- The Data Access Layer retrieves the object from the OODBMS.
- The information is returned to the Business Layer.
- The Presentation Layer displays the stock information.

This process keeps database operations separate from the user interface and business logic.

Inventory Threshold Example

The Observer Pattern is particularly useful for warehouse management.

Suppose the minimum stock level for a product is **10 units**.

If the current stock becomes:

Stock = 8 units

the system detects that the minimum threshold has been reached.

The system can then automatically generate:

Inventory Alert → Warehouse Manager

The manager can create a purchase order to replenish the stock.

This reduces the possibility of stock-outs and improves inventory management.

Flexibility, Scalability and Maintainability

Requirement	Contribution
Flexibility	New product categories and business rules can be added easily
Scalability	The layered architecture allows new retail services to be integrated
Maintainability	Each layer has a separate responsibility
Reusability	Common product and business components can be reused
Efficiency	DAO provides organized access to persistent inventory objects
Automation	Observer Pattern automatically alerts managers about low stock
Extensibility	Future stock prediction and robotic warehouse management can be added

Future Enhancements

The architecture is designed to support future integration with **automated stock prediction and robotic warehouse management**.

Automated stock prediction can analyze inventory levels and help determine when new stock should be ordered.

Robotic warehouse management can be integrated with the existing system to automate activities such as product movement, stock checking, and warehouse operations.

Because the system uses layered architecture and design patterns, these future features can be added without completely redesigning the existing application.

Overall Working

The overall working of the system can be summarized as:

Customer/Manager



Presentation Layer / MVC



Business Layer



Domain Objects



DAO / Data Access Layer



OODBMS

At the same time, the **Factory Pattern** creates product objects and the **Observer Pattern** monitors inventory levels and sends alerts when the stock reaches the minimum threshold.

Conclusion

The Smart Retail and Inventory Management System demonstrates how **object-oriented principles, layered architecture, and design patterns** can be used to build a flexible and maintainable retail application.

Encapsulation protects product and inventory data, abstraction reduces complexity, inheritance provides reuse, and polymorphism allows different product categories to have specialized behaviour.

The MVC Pattern separates the user interface from application logic, the Factory Pattern simplifies product creation, the Observer Pattern provides automatic low-stock notifications, and the DAO Pattern manages communication with persistent inventory objects.

The layered architecture provides a clear path between the user interface, business operations, domain objects, data access, and OODBMS. Therefore, the proposed system is **flexible, scalable, maintainable, efficient, and ready for future enhancements** such as automated stock prediction and robotic warehouse management.

3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

ANSWER:

Introduction

The Smart Hotel Management System is designed to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The system follows a **layered object-oriented architecture** consisting of the Presentation Layer, Service Layer, Domain Layer, and Data Access Layer connected to an **Object-Oriented Database Management System (OODBMS)**.

The system uses the **Strategy Pattern, Factory Pattern, Observer Pattern, and Repository Pattern**. These patterns improve flexibility, reusability, maintainability, and efficient management of hotel operations.

Object-Oriented Principles

Encapsulation

Encapsulation combines data and operations within a class and protects important hotel information.

For example, a Guest class can contain guest name, guest ID, contact details, and booking information. Methods can be used to update or retrieve this information in a controlled manner.

Similarly, a Room class can contain room number, room type, availability, and price.

This prevents direct and unauthorized modification of important hotel data.

Abstraction

Abstraction hides complex internal operations from users.

For example, when a guest makes a reservation, the guest only provides the required booking details. The internal processes such as checking room availability, calculating the price, updating the reservation, and storing the information are handled by the system.

This makes the application easier to use and reduces complexity.

Inheritance

Inheritance allows common properties and methods to be reused.

For example:

Room → **StandardRoom**

Room → **DeluxeRoom**

Room → **SuiteRoom**

Common properties such as room number, availability, and basic price can be defined in the Room class, while specialized room types can have additional features.

This improves code reuse and reduces duplication.

Polymorphism

Polymorphism allows different room or service objects to respond differently to the same operation.

For example, a calculatePrice() operation can produce different results for a standard room, deluxe room, and suite depending on their pricing rules.

This makes the system flexible when new room types or services are introduced.

Layered Architecture

The hotel system consists of four major layers.

Layer	Responsibility
Presentation Layer	Provides the interface for guests and hotel employees
Service Layer	Processes reservations, payments, check-in/check-out and services
Domain Layer	Represents guests, rooms, reservations, payments and services
Data Access Layer	Communicates with the OODBMS and manages persistent hotel objects

Presentation Layer

The Presentation Layer provides the user interface for guests and hotel staff.

It can provide functions such as:

- Guest registration
- Room search
- Room reservation
- Check-in
- Check-out
- Payment
- Room service requests
- Customer feedback

The presentation layer sends requests to the Service Layer.

Service Layer

The Service Layer coordinates the main hotel operations.

It handles activities such as:

- Checking room availability
- Processing reservations
- Calculating booking charges
- Processing payments
- Managing check-in and check-out
- Coordinating room services

This layer ensures that hotel business rules are correctly applied.

Domain Layer

The Domain Layer contains the main hotel objects.

Examples include:

- Guest

- Room
- Reservation
- Payment
- RoomService
- Feedback

The domain objects represent the real-world entities of the hotel system.

Data Access Layer

The Data Access Layer communicates with the OODBMS.

It is responsible for operations such as:

- Saving guest objects
- Storing room information
- Saving reservations
- Updating room availability
- Storing payment information
- Retrieving customer feedback

This keeps database operations separate from the business logic.

Design Patterns Used

Strategy Pattern

The Strategy Pattern is used for different **pricing and discount policies**.

Different pricing strategies can be created for different situations.

For example:

Pricing Strategy

→ Standard Pricing

→ Weekend Pricing

→ Seasonal Pricing

Similarly, discount strategies can be applied for:

- Regular customers
- Premium customers
- Promotional offers

The strategy can be changed without modifying the main reservation-processing code.

Benefits:

- Provides flexible pricing
- Makes discount policies interchangeable
- Supports future pricing rules
- Reduces changes to existing code

Factory Pattern

The Factory Pattern is used for creating different **room and service objects**.

For example:

Room Factory → **Standard Room / Deluxe Room / Suite**

A Service Factory can create:

Service Factory → **Food Service / Laundry Service / Cleaning Service**

The factory handles object creation instead of creating objects directly throughout the application.

Benefits:

- Simplifies object creation
- Reduces dependency between classes
- Supports new room and service types
- Improves maintainability

Observer Pattern

The Observer Pattern is used for **reservation and service notifications**.

For example, when a reservation is confirmed:

Reservation → Notification System → Guest

The guest can receive a confirmation notification.

Similarly, when a room-service request is updated:

Room Service → Notification System → Guest/Hotel Staff

This allows notifications to be generated automatically.

Benefits:

- Supports real-time notifications
- Improves guest communication
- Reduces manual notification work
- Allows additional notification services to be added

Repository Pattern

The Repository Pattern simplifies communication between business components and persistent hotel objects.

For example, a RoomRepository can provide operations such as:

- saveRoom()
- findRoom()
- updateRoom()
- deleteRoom()

Similarly, a ReservationRepository can manage reservation objects.

The Service Layer can use these repositories without directly interacting with the OODBMS.

Benefits:

- Separates database operations from business logic
- Simplifies data access
- Improves maintainability

- Makes the system easier to test

Reservation Processing

Reservation processing demonstrates how the different layers work together.

The overall flow is:

Guest → Presentation Layer → Service Layer → Domain Layer → Repository → Data Access Layer → OODBMS

When a guest makes a reservation:

- The guest enters the required details.
- The Presentation Layer sends the request to the Service Layer.
- The Service Layer checks room availability.
- The Domain Layer applies reservation rules.
- The Strategy Pattern calculates the appropriate room price or discount.
- The Factory Pattern can create the required room or service object.
- The Repository manages the reservation object.
- The Data Access Layer stores the object in the OODBMS.
- The Observer Pattern sends a reservation confirmation to the guest.

This provides a systematic and organized reservation process.

Communication with the OODBMS

The OODBMS provides persistent storage for hotel-related objects.

Storing Hotel Objects

When a new reservation is created:

- The reservation details are received from the Presentation Layer.
- The Service Layer validates the request.
- The Domain Layer creates or updates the required objects.
- The Repository passes the object to the Data Access Layer.
- The Data Access Layer stores the persistent object in the OODBMS.

The same process can be used to store guests, rooms, payments, and room-service information.

Retrieving Hotel Objects

When the system needs information:

- The Service Layer sends a request to the Repository.
- The Repository communicates with the Data Access Layer.
- The Data Access Layer retrieves the required object from the OODBMS.
- The object is returned to the Service Layer.
- The result is displayed through the Presentation Layer.

For example, when a guest searches for available rooms, the system retrieves room objects and checks their availability before displaying the available rooms.

FLEXIBILITY, REUSABILITY AND MAINTAINABILITY

Factor	Contribution
Flexibility	Pricing and discount strategies can be changed without modifying the complete system
Reusability	Common room, guest, reservation and service components can be reused
Maintainability	Each layer has a clearly defined responsibility
Scalability	New hotel services and room types can be added
Extensibility	New smart-room and personalized services can be integrated
Automation	Observer Pattern provides automatic reservation and service notifications

Example of Check-In and Check-Out

The layered architecture can also be used for check-in and check-out operations.

During **check-in**:

Guest → Presentation Layer → Service Layer → Domain Layer → Repository → OODBMS

The system verifies the reservation, updates the room status, and stores the updated room and guest information.

During **check-out**, the system calculates the final payment using the appropriate pricing strategy, processes the payment, updates room availability, and stores the updated information in the OODBMS.

This ensures that room availability and payment information remain consistent.

Future Enhancements

The system is designed to support future enhancements such as **smart-room automation and personalized guest services**.

Smart-room automation can allow hotel systems to interact with room devices, while personalized services can provide customized recommendations and services based on guest requirements.

The existing layered architecture and design patterns make it possible to add these features without redesigning the complete system.

Conclusion

The Smart Hotel and Hospitality Management System demonstrates how **object-oriented principles, layered architecture, and design patterns** can be used to develop a flexible and maintainable hotel application.

Encapsulation protects guest and hotel data, abstraction reduces complexity, inheritance provides code reuse, and polymorphism allows different room and service types to behave appropriately.

The Strategy Pattern provides flexible pricing and discount policies, the Factory Pattern simplifies the creation of room and service objects, the Observer Pattern provides reservation and service notifications, and the Repository Pattern simplifies communication with persistent hotel objects.

The layered architecture provides a clear separation between the user interface, hotel services, domain objects, and data access. The OODBMS provides persistent storage for hotel information.

Therefore, the proposed system provides a **flexible, reusable, maintainable, scalable, and extensible solution** for modern hotel management and can support future features such as smart-room automation and personalized guest services.

4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

ANSWER:

Introduction

The Smart Logistics and Delivery Management System is designed to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer

notifications. The system follows a **layered object-oriented architecture** consisting of the Presentation Layer, Business Layer, Domain Layer, and Data Access Layer connected to an **Object-Oriented Database Management System (OODBMS)**.

The system uses the **Strategy Pattern, Factory Pattern, Observer Pattern, and DAO Pattern**. These design patterns help improve the adaptability, scalability, maintainability, and efficiency of logistics operations.

Object-Oriented Design Principles

Encapsulation

Encapsulation combines data and operations inside classes and protects important logistics information.

For example, a Shipment class can contain shipment ID, source, destination, weight, status, and delivery details. Methods such as `updateStatus()` and `assignVehicle()` can control how shipment information is modified.

Similarly, a Vehicle class can contain vehicle number, type, capacity, and availability.

This prevents unauthorized or incorrect modification of important logistics data.

Abstraction

Abstraction hides complex implementation details and provides only the required operations.

For example, a logistics employee can select **Assign Vehicle** without knowing the internal process used to check vehicle capacity, availability, and shipment requirements.

This reduces system complexity and makes the application easier to use.

Inheritance

Inheritance allows common properties and operations to be reused among related classes.

For example:

Shipment → StandardShipment
Shipment → ExpressShipment
Shipment → InternationalShipment

Similarly:

Vehicle → Truck
Vehicle → Van
Vehicle → DeliveryVehicle

Common properties can be defined in parent classes, while specialized behaviour can be implemented in child classes.

Polymorphism

Polymorphism allows the same operation to behave differently depending on the object.

For example, different shipment types can use different delivery or pricing rules while using a common shipment interface.

Similarly, different vehicles can have different capacity or allocation rules.

This makes the system easier to extend when new shipment or vehicle types are introduced.

Layered Architecture

The logistics system consists of four major layers.

Layer	Responsibility
Presentation Layer	Provides interfaces for logistics employees, managers and customers
Business Layer	Processes shipment, warehouse, vehicle and delivery operations
Domain Layer	Represents shipments, vehicles, warehouses and delivery information
Data Access Layer	Communicates with the OODBMS and manages persistent objects

Presentation Layer

The Presentation Layer provides the interface through which users interact with the logistics system.

It can provide functions such as:

- Registering shipments
- Viewing shipment status
- Allocating vehicles
- Managing warehouse operations
- Tracking deliveries
- Viewing delivery notifications

The Presentation Layer sends requests to the Business Layer.

Business Layer

The Business Layer manages the main logistics operations.

It handles activities such as:

- Shipment registration
- Vehicle allocation
- Warehouse management
- Route selection
- Delivery tracking
- Customer notifications

It applies business rules and coordinates the different components of the system.

Domain Layer

The Domain Layer contains the main logistics objects.

Examples include:

- Shipment
- Vehicle
- Warehouse
- Delivery

- Customer
- DeliveryStatus

These objects represent the real-world entities involved in logistics operations.

Data Access Layer

The Data Access Layer communicates with the OODBMS.

It performs operations such as:

- Storing shipment objects
- Retrieving vehicle information
- Updating delivery status
- Saving warehouse information
- Retrieving customer delivery information

This keeps database operations separate from the business logic.

Design Patterns Used

Strategy Pattern

The Strategy Pattern is used for **selecting optimal delivery routes**.

Different route-selection strategies can be used depending on the requirements.

For example:

Route Strategy

→ Shortest Route

→ Fastest Route

→ Cost-Optimized Route

The appropriate strategy can be selected based on traffic, distance, delivery time, or cost.

Benefits:

- Provides flexible route selection
- Allows algorithms to be changed easily
- Supports different delivery requirements
- Makes future AI-based optimization easier

Factory Pattern

The Factory Pattern is used for creating different **shipment types**.

For example:

Shipment Factory → Standard Shipment / Express Shipment / Special Shipment

The factory creates the required shipment object based on the user's requirements.

Benefits:

- Simplifies object creation
- Reduces dependency between classes
- Supports new shipment types
- Improves maintainability

Observer Pattern

The Observer Pattern is used for **real-time delivery status notifications**.

For example:

Delivery Status → Notification Service → Customer

When the shipment status changes, the customer can automatically receive a notification.

For example:

Shipment Registered → In Transit → Out for Delivery → Delivered

The customer can receive an update whenever the status changes.

Benefits:

- Provides real-time notifications
- Improves customer experience
- Reduces manual communication
- Supports multiple notification services

DAO Pattern

The **Data Access Object (DAO) Pattern** manages persistent shipment and vehicle information.

For example:

ShipmentDAO

can provide:

- saveShipment()
- findShipment()
- updateShipment()
- deleteShipment()

Similarly, VehicleDAO can manage vehicle information.

The Business Layer can access these operations without directly communicating with the OODBMS.

Benefits:

- Separates database access from business logic
- Improves maintainability
- Simplifies database operations
- Makes testing easier

Communication with the OODBMS

The system uses the following communication flow:

User → Presentation Layer → Business Layer → Domain Layer → DAO/Data Access Layer → OODBMS

Storing Shipment Information

When a new shipment is registered:

- The user enters shipment details through the Presentation Layer.
- The Business Layer validates the shipment information.
- The Factory Pattern creates the appropriate shipment object.
- The Domain Layer applies the required logistics rules.
- The DAO passes the object to the Data Access Layer.
- The Data Access Layer stores the persistent shipment object in the OODBMS.

Retrieving Shipment Information

When a customer wants to track a shipment:

- The customer sends a tracking request.
- The Presentation Layer forwards the request.
- The Business Layer processes the request.
- The DAO requests the shipment object.
- The Data Access Layer retrieves it from the OODBMS.
- The shipment status is returned to the Presentation Layer.
- The customer can view the current delivery status.

Vehicle Allocation

Vehicle allocation is another important operation.

When a shipment requires a vehicle:

- The system retrieves available vehicle objects from the OODBMS.
- The Business Layer checks vehicle availability and suitability.
- The shipment requirements are compared with vehicle capacity.
- An appropriate vehicle is allocated.
- The vehicle information is updated.
- The updated vehicle object is stored back in the OODBMS.

This ensures that shipment and vehicle information remains consistent.

Delivery Tracking and Notifications

The Observer Pattern is useful for real-time tracking.

For example:

Shipment Created

↓

Shipment Dispatched

↓

In Transit

↓

Out for Delivery

↓

Delivered

Whenever the delivery status changes, the Observer Pattern can automatically notify the customer.

This improves transparency and allows customers to track their shipments without repeatedly contacting the logistics company.

Adaptability, Scalability and Maintainability

Requirement	Contribution of the Design
Adaptability	Different route-selection strategies can be introduced easily
Scalability	New shipment, vehicle and delivery services can be added
Maintainability	Each layer has a clearly defined responsibility
Flexibility	Factory and Strategy Patterns allow components to be changed easily

Reusability	Common shipment and vehicle components can be reused
Real-Time Operation	Observer Pattern provides delivery status notifications
Data Management	DAO provides organized access to persistent logistics information

Data Management DAO provides organized access to persistent logistics information

Future Enhancements

The architecture is designed to support future integration with **autonomous delivery vehicles and AI-based route optimization**.

AI-based route optimization can analyze delivery requirements and determine suitable routes based on factors such as travel time and route efficiency.

Autonomous delivery vehicles can also be integrated as new vehicle types without changing the complete system.

The Strategy Pattern is particularly useful for introducing advanced route-selection algorithms, while the Factory Pattern can support new autonomous vehicle objects.

Overall Working

The complete working of the system can be represented as:

Customer / Logistics Employee

↓

Presentation Layer

↓

Business Layer

↓

Domain Objects



DAO / Data Access Layer



OODBMS

The **Factory Pattern** creates shipment objects, the **Strategy Pattern** selects the appropriate delivery route, and the **Observer Pattern** sends real-time delivery notifications.

Conclusion

The Smart Logistics and Delivery Management System demonstrates how **object-oriented principles, layered architecture, and design patterns** can be effectively applied to solve real-world logistics problems.

Encapsulation protects shipment and vehicle information, abstraction reduces complexity, inheritance improves code reuse, and polymorphism provides flexibility for different shipment and vehicle types.

The Strategy Pattern provides flexible route selection, the Factory Pattern simplifies shipment creation, the Observer Pattern provides real-time delivery notifications, and the DAO Pattern manages persistent shipment and vehicle information.

The layered architecture separates the presentation, business, domain, and data access responsibilities, while the OODBMS provides persistent storage for logistics objects.

Therefore, the proposed system provides an **adaptable, scalable, maintainable, flexible, and efficient logistics solution** that can support future technologies such as autonomous delivery vehicles and AI-based route optimization.

5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the

presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

ANSWER:

Introduction

The Smart Energy Management System is designed to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a **layered object-oriented architecture** consisting of the Presentation Layer, Business Layer, Domain Layer, and Data Access Layer connected to an **Object-Oriented Database Management System (OODBMS)**.

The system uses the **Observer Pattern, Strategy Pattern, Factory Pattern, and Repository Pattern**. These patterns improve the scalability, extensibility, flexibility, and maintainability of the energy management system.

Object-Oriented Principles

Encapsulation

Encapsulation combines data and operations within a class and protects energy-related information.

For example, a SmartMeter class can contain meter ID, location, consumption value, meter type, and status. Methods such as recordConsumption() and updateReading() can control how meter information is modified.

This protects important energy-monitoring data and prevents incorrect direct modification.

Abstraction

Abstraction hides complex implementation details and provides only the required functionality to users.

For example, a user can view electricity consumption through a dashboard without knowing how the system collects meter readings, analyzes consumption, or stores the information in the OODBMS.

This reduces complexity and makes the system easier to operate.

Inheritance

Inheritance allows common properties and operations to be reused between related smart-meter classes.

For example:

SmartMeter → **ResidentialMeter**

SmartMeter → **CommercialMeter**

SmartMeter → **IndustrialMeter**

Common properties such as meter ID, location, reading, and status can be defined in the SmartMeter class, while specialized behaviour can be implemented in the derived classes.

This reduces code duplication and improves reusability.

Polymorphism

Polymorphism allows different smart-meter objects to perform the same operation in different ways.

For example, the method `calculateConsumption()` can work differently depending on the type of smart meter.

This allows the system to support different meter types without changing the complete application.

Layered Architecture

The system consists of four major layers.

Layer	Responsibility
Presentation Layer	Provides dashboards and interfaces for monitoring energy usage
Business Layer	Performs energy analysis and manages energy-related operations
Domain Layer	Contains energy rules and represents smart meters and consumption objects
Data Access Layer	Communicates with the OODBMS and manages persistent energy data

Presentation Layer

The Presentation Layer provides dashboards for viewing energy information.

Users can view:

- Electricity consumption
- Smart meter readings
- Energy usage analysis
- Abnormal consumption alerts
- Historical energy information

The dashboard receives processed information from the Business Layer.

Business Layer

The Business Layer performs the main energy-analysis operations.

It can:

- Analyze electricity consumption
- Detect abnormal usage
- Apply energy optimization algorithms
- Generate energy-related reports
- Coordinate smart-meter information

This layer applies the required business rules before information is presented to users.

Domain Layer

The Domain Layer contains the main energy-related objects and rules.

Examples include:

- SmartMeter
- EnergyConsumption
- EnergyReading
- EnergyAlert
- EnergyUsage

This layer represents the actual energy-management concepts used by the system.

Data Access Layer

The Data Access Layer communicates with the OODBMS.

It performs operations such as:

- Storing smart-meter objects
- Saving energy readings
- Retrieving historical consumption
- Updating meter information
- Storing abnormal consumption data

This keeps database operations separate from energy-analysis logic.

Design Patterns Used

Observer Pattern

The Observer Pattern is used for **consumption alerts**.

When abnormal electricity consumption is detected, the system can automatically notify the required users.

For example:

Smart Meter → Abnormal Consumption → Alert System → User

If the consumption exceeds a predefined level, an alert can be generated automatically.

Benefits:

- Provides real-time alerts
- Helps detect abnormal consumption
- Reduces manual monitoring
- Improves energy awareness
- Allows multiple users or services to receive alerts

Strategy Pattern

The Strategy Pattern is used for different **energy optimization algorithms**.

Different optimization strategies can be used depending on the energy requirements.

For example:

Energy Optimization Strategy

→ Cost Reduction Strategy

→ Peak Usage Reduction Strategy

→ Energy Efficiency Strategy

The system can select the appropriate strategy without changing the complete energy-analysis system.

Benefits:

- Provides flexibility
- Allows optimization algorithms to be changed easily
- Supports different energy requirements
- Makes future AI-based optimization easier

Factory Pattern

The Factory Pattern is used for creating different **smart-meter objects**.

For example:

Smart Meter Factory → Residential Meter / Commercial Meter / Industrial Meter

The Factory determines which smart-meter object needs to be created.

Benefits:

- Simplifies object creation
- Reduces dependency between classes
- Supports new smart-meter types
- Improves maintainability

Repository Pattern

The Repository Pattern is used to manage persistent energy data.

For example, an EnergyRepository can provide operations such as:

- saveReading()
- findReading()
- updateReading()
- deleteReading()

The Business Layer can use the repository without directly communicating with the OODBMS.

Benefits:

- Separates data access from business logic

- Simplifies database operations
- Improves maintainability
- Makes testing easier

Interaction with the OODBMS

The different system layers communicate with the OODBMS through the Data Access Layer and Repository Pattern.

The overall flow is:

User → Presentation Layer → Business Layer → Domain Layer → Repository → Data Access Layer → OODBMS

Storing Energy Data

When a smart meter sends a new energy reading:

- The reading is received by the system.
- The Business Layer processes the reading.
- The Domain Layer represents the reading as an energy object.
- The Repository manages the persistent object.
- The Data Access Layer communicates with the OODBMS.
- The energy-reading object is stored in the database.

This allows historical energy information to be maintained.

Retrieving Energy Data

When a user wants to view energy consumption:

- The user requests the information through the dashboard.
- The Presentation Layer sends the request to the Business Layer.
- The Business Layer requests the required data.
- The Repository accesses the Data Access Layer.
- The Data Access Layer retrieves the persistent energy objects from the OODBMS.
- The information is returned to the Business Layer.
- The Presentation Layer displays the results on the dashboard.

This provides efficient access to both current and historical energy information.

Abnormal Consumption Detection

The system can use the Observer Pattern to generate alerts when abnormal consumption occurs.

For example, if normal electricity consumption is within a particular range and the smart meter suddenly reports unusually high consumption:

Smart Meter Reading

↓

Business Layer Analysis

↓

Abnormal Consumption Detected

↓

Observer Pattern

↓

Alert Generated

↓

User Notification

This allows users to identify unusual energy usage quickly.

Energy Optimization

The Strategy Pattern allows the system to use different energy optimization algorithms.

For example, the system can select an optimization strategy based on the current requirement.

Strategy	Purpose
Cost Reduction	Reduces energy-related cost

Peak Usage Reduction	Reduces consumption during high-demand periods
Energy Efficiency	Improves overall energy usage
AI-Based Strategy	Predicts and recommends optimized energy consumption

The strategy can be changed without modifying the main energy-management system.

Scalability, Extensibility and Maintainability

Requirement	Contribution of the Design
Scalability	New energy services and monitoring features can be added
Extensibility	New smart-meter types and optimization algorithms can be introduced
Flexibility	Strategy Pattern allows different optimization methods
Maintainability	Each layer has a separate responsibility
Reusability	Common energy and meter components can be reused
Automation	Observer Pattern automatically generates consumption alerts
Data Management	Repository Pattern provides organized access to persistent energy data

Future Enhancements

The architecture is designed to support **renewable-energy monitoring and AI-based consumption prediction**.

Renewable-energy monitoring can be added to track energy generated from sources such as solar or other renewable systems.

AI-based consumption prediction can analyze historical energy readings and help predict future consumption. The prediction results can then be used with the Strategy Pattern to recommend suitable energy-optimization methods.

These features can be added without completely redesigning the existing system because the architecture separates different responsibilities.

Overall Working

The overall working of the Smart Energy Management System can be represented as:

Smart Meter / User



Presentation Layer



Business Layer



Domain Layer



Repository / Data Access Layer



OODBMS

The **Factory Pattern** creates smart-meter objects, the **Strategy Pattern** selects energy-optimization algorithms, the **Observer Pattern** generates consumption alerts, and the **Repository Pattern** manages persistent energy data.

Conclusion

The Smart Energy Management System demonstrates how **object-oriented principles, layered architecture, and design patterns** can be combined to develop a scalable and extensible energy-management solution.

Encapsulation protects energy and meter data, abstraction reduces complexity, inheritance improves code reuse, and polymorphism allows different smart-meter types to provide specialized behaviour.

The Observer Pattern provides automatic consumption alerts, the Strategy Pattern supports different energy-optimization algorithms, the Factory Pattern simplifies smart-meter object creation, and the Repository Pattern manages persistent energy data.

The layered architecture separates the Presentation, Business, Domain, and Data Access responsibilities, while the OODBMS provides persistent storage for energy-monitoring objects.

Therefore, the proposed system provides a **scalable, flexible, extensible, maintainable, and efficient solution** for modern energy management. It can also be extended in the future to support **renewable-energy monitoring and AI-based consumption prediction**.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation